

VASP 추상화 구조

IVASPAdapter · ExternalVASPAdapter · Phase 3 전환 설계

VASP 연동 + 복구 + 멀티체인 추상화

Strategy Pattern

DI 교체

internal/packages/vasp

왜 VASP를 인터페이스로 추상화하는가

VENDOR LOCK-IN 방지 — PHASE 전환 비용 최소화

문제 — 직접 호출 시

IssuerService → VASP REST API 직접 호출

Phase 3에서 교보 자체 VASP 취득 시:

- IssuerService 코드 **전체 수정**
- 테스트 교체, 배포 리스크 증가

규제 맥락 (특금법)

가상자산사업자(VASP) 신고·인가 없이 직접 운영 불가

→ Phase 1: 인가받은 외부 VASP에 TX 위탁

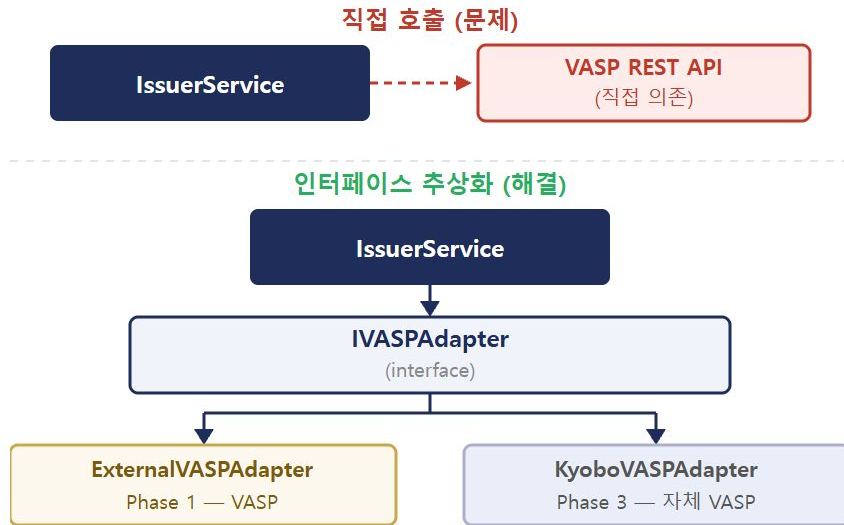
→ Phase 3: 교보생명 직접 VASP 인가 취득 시 내재화

해결 — DI 교체 1줄

IssuerService는 **IVASPAAdapter** 만 알면 됨

Phase 전환 = DI 바인딩 교체만으로 완료

IssuerService 코드 수정 없음



IVASPAAdapter 인터페이스 (1/2)

INTERFACE DECLARATION — 6 METHODS

internal/packages/vasp/src/interfaces/IVASPAAdapter.ts

```
export interface IVASPAAdapter {  
  
  // Phase 1 핵심 write 경로  
  // VASP가 TX 서명·브로드캐스트  
  submitTransaction(  
    params: SubmitTransactionParams  
  ): Promise<VASPTransactionReceipt>;  
  
  // 수탁 지갑 생성/조회  
  createWallet(userId: string): Promise<WalletInfo>;  
  getWallet(userId: string): Promise<WalletInfo | null>;  
  
  // 토큰 전송 (Travel Rule 포함)  
  transfer(req: TransferRequest): Promise<TransferResult>;  
  
  // TX 상태 조회 (pollStaleRequests 사용)  
  getTransferStatus(  
    txHash: string  
  ): Promise<TransferResult>;  
  
  // AML 스크리닝 - 블랙리스트 주소 여부  
  screenAddress(  
    address: string  
  ): Promise<{ flagged: boolean; reason?: string }>;  
}
```

메서드	Phase	역할
submitTransaction	1	TX 위탁 — 핵심 write 경로
createWallet	1	수탁 지갑 생성
getWallet	1	지갑 주소 조회
transfer	1	토큰 전송 + Travel Rule
getTransferStatus	1	TX 상태 조회 (polling)
screenAddress	1	AML 블랙리스트 체크

왜 6개인가 — VASP는 TX 전송뿐 아니라 지갑 관리(custody) + 규제 의무(Travel Rule, AML)까지 포함. 이 범위 전체를 하나의 인터페이스로 추상화.

IVASPAdapter 인터페이스 (2/2) — 핵심 타입

TYPE DEFINITIONS — SUBMIT PARAMS · TRAVEL RULE · RECEIPT

```
interface SubmitTransactionParams {
  contractAddr: string;
  abi: unknown[];
  method: string;
  args: unknown[];
  idempotencyKey: string; // 중복 TX 방지
}

interface TravelRuleData {
  originator: { name: string; accountId: string; vasp: string; };
  beneficiary: { name: string; accountId: string; vasp: string; };
}

// submitTransaction() 반환
interface VASPTransactionReceipt {
  txHash: string;
  status: 'submitted' | 'pending' | 'confirmed' | 'failed';
  timestamp: number;
}

// transfer() / getTransferStatus() 반환
interface TransferResult {
  txHash?: string;
  status: 'pending' | 'completed' | 'failed';
  // completed → 내부 confirmed로 변환
}
```

idempotencyKey — 역등성 키

같은 키로 두 번 요청하면 VASP가 두 번째를 무시
네트워크 재시도 시 중복 TX 방지
값: `activityId`, `requestId` 등 비즈니스 고유 키

Travel Rule — 특금법 §8의4

100만원 이상 이체 시 수신자 정보를
수취 VASP에 암호화 전송 의무
미이행 시 과태료 + 영업정지 리스크

두 타입의 status가 다른 이유

`VASPTransactionReceipt` — TX 브로드캐스트 단계 (4단계)
`TransferResult` — 이체 완료 여부 (3단계, `completed`)
`VaspTxClientAdapter` 가 `completed` → `confirmed` 변환

ExternalVASPAdapter — 역할 및 보안 요구사항

PHASE 1 IMPLEMENTATION — EXTERNAL VASP REST API

internal/packages/vasp/src/external/ExternalVASPAdapter.ts

```
export class ExternalVASPAdapter
  implements IVASPAdapter {
  private readonly baseUrl: string;
  private readonly apiKey: string;
  constructor(config: {
    baseUrl: string;
    apiKey: string;
  }) {
    this.baseUrl = config.baseUrl;
    this.apiKey = config.apiKey;
  }
  // _request() - 내부 HTTP 클라이언트
  private async _request(
    method, path, body?
  ): Promise<Record<string, unknown>> {
    const res = await fetch(
      `${this.baseUrl}${path}`,
      { method, headers: { 'X-API-Key': this.apiKey },
        body: JSON.stringify(body) });
    if (!res.ok) throw new Error(`VASP API ${res.status}: ${path}`);
    return res.json();
  }
}
```

역할

외부 인가 VASP REST API를 호출하는 Phase 1 구현체. 파트너사가 바뀌어도 이 클래스만 수정 — `IVASPAdapter` 인터페이스는 유지.

보안 요구사항

- 1 API 키 주입 — 환경 변수 또는 KMS. 코드 하드코딩 절대 금지
- 2 내부망 경유 — 모든 호출은 방화벽을 통해 외부 VASP API로만 허용
- 3 Travel Rule 강제 — 100만원 이상 이체 시 `travelRuleData` 필수 포함

ExternalVASPAdapter — 핵심 메서드

SUBMIT + STATUS + TRAVEL RULE — PHASE 1 WRITE PATH

submitTransaction — Phase 1 write 핵심

```
async submitTransaction(  
  params: SubmitTransactionParams  
) : Promise<VASPTransactionReceipt> {  
  const res = await this._request('POST', '/transactions', {  
    contractAddr: params.contractAddr,  
    method: params.method,  
    args: params.args,  
    idempotencyKey: params.idempotencyKey,  
  });  
  return { txHash: res['txHash'] as string,  
    status: res['status'] as VASPTransactionReceipt['status'],  
    timestamp: (res['timestamp'] as number) ?? Date.now() };  
}
```

getTransferStatus — TX 상태 polling

```
async getTransferStatus(  
  txHash: string  
) : Promise<TransferResult> {  
  const res = await this._request('GET', `~/transfers/${txHash}`);  
  return { txHash: ..., status: ..., fee: BigInt(...) };  
}
```

ABI는 전달하지 않는다

`submitTransaction` 에 `abi` 가 있지만 VASP API 전송 payload에는 포함 안 함. VASP가 직접 컨트랙트 ABI를 보유·사용. 우리는 method 이름과 args만 전달.

AML — screenAddress

GET /aml/screen/:address

flagged: true → 블랙리스트

IssuerService가 mintNFT 전 호출

→ 제재 주소로의 발행 사전 차단

transfer + Travel Rule

```
if (req.travelRuleData) {  
  payload['travelRule'] = req.travelRuleData;  
  // 100만원↑ 이체: 수신자 VASP에  
  // 암호화 채널로 originator 정보 전달  
}
```

VaspTxClientAdapter (1/2) — 왜 필요한가

BRIDGE LAYER — INTERFACE LANGUAGE TRANSLATION

internal/packages/vasp/src/tx/VaspTxClientAdapter.ts



VaspTxClient — TX 실행 인터페이스

메서드

역할

`submitMint()`

NFT 발행 TX 제출

`getStatus()`

TX 상태 조회

`resubmitWithGasBump()`

gas 인상 재전송 (Phase 3)

TxStateMachineService가 요구하는 언어 — TX 실행에 특화된 낮은 레벨 인터페이스

IVASPAAdapter — 비즈니스 인터페이스

메서드

역할

`submitTransaction()`

TX 위탁 (일반 용어)

`getTransferStatus()`

이체 상태 조회

`createWallet / transfer`

지갑 관리 / 전송

비즈니스 언어 — 더 높은 레벨 — VASP 서비스 전체를 표현. SRP: 하나의 역할 = 하나의 인터페이스

VaspTxClientAdapter (2/2) — 핵심 메서드

LANGUAGE TRANSLATION — VASP STATUS → STATE MACHINE STATUS

submitMint → submitTransaction 변환

```
async submitMint(params: {
  to: string; tokenId: bigint;
  amount: bigint; requestId: string;
}): Promise<{ txHash: string }> {
  const receipt = await this.vasp.submitTransaction({
    contractAddr: this.nftIssuerAddr,
    abi: NFT_ISSUER_ABI,
    method: 'mint',
    args: [params.to, params.tokenId.toString(),
      params.amount.toString(),
      `0x${Buffer.from(params.requestId)...}`,
    ],
    idempotencyKey: params.requestId,
  });
  if (receipt.status === 'failed')
    throw new Error(`VASP TX failed: ${receipt.txHash}`);
  return { txHash: receipt.txHash };
}
```

getStatus — 상태어 변환

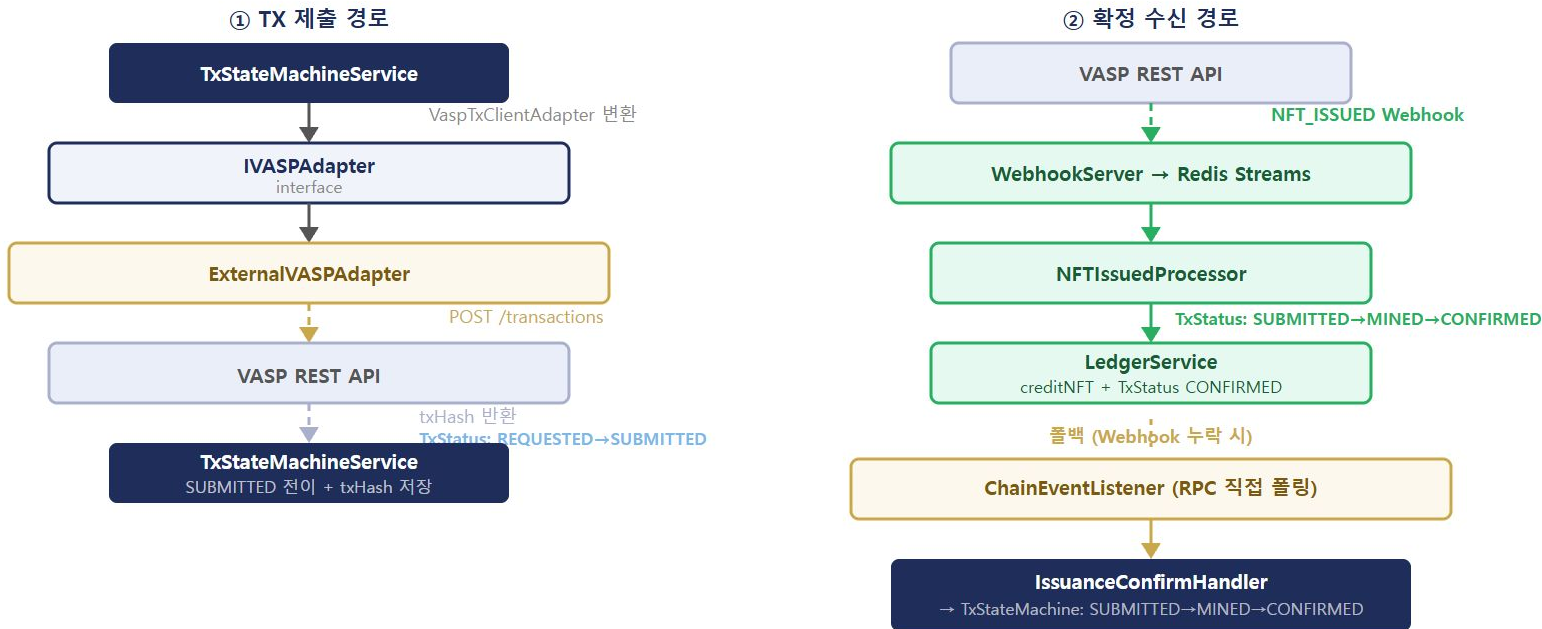
```
async getStatus(txHash: string): Promise<{
  status: 'pending' | 'mined' | 'confirmed'
  | 'failed' | 'not_found';
}> {
  const result = await this.vasp.getTransferStatus(txHash);
  // VASP 상태어 → 내부 상태머신 언어 변환
  const status =
    result.status === 'completed' ? 'confirmed'
    : result.status === 'pending' ? 'pending'
    : 'failed';
  return { status };
}

// Phase 3에서 구현 예정
async resubmitWithGasBump(): Promise<...> {
  throw new Error('Phase 3에서 구현 예정');
}
```

상태어 변환이 핵심 — VASP의 `completed` 를 내부의 `confirmed` 로 번역.
TxStateMachineService는 VASP 언어를 모른다.

레이어 전체 조립도

FULL LAYER ASSEMBLY — PHASE 1 WRITE PATH



Phase 3 전환 시 — ExternalVASPADapter → KyoboVASPADapter (DI 바인딩 1줄 교체) / ① 정상 경로 ② 풀백 모두 Phase 1부터 공존

KyoboVASPAdapter (1/2) — 현재 상태 및 교체 범위

PHASE 3 STUB — ALL METHODS THROW

internal/packages/vasp/src/internal/KyoboVASPAdapter.ts

```
export class KyoboVASPAdapter
  implements IVASPAdapter {

  async submitTransaction(...) {
    throw new Error(
      'KyoboVASPAdapter: not implemented'
    );
  }

  async createWallet(...) {
    throw new Error('...');
  }

  async getWallet(...) { throw ... }
  async transfer(...) { throw ... }
  async getTransferStatus(...) { throw ... }
  async screenAddress(...) { throw ... }
}
```

모든 메서드 throw — 향후 내재화 시까지 호출 불가. TypeScript 컴파일은 통과 (인터페이스 계약 준수 확인용 존재)

교체 범위 — index.ts 1줄만

```
// Phase 1 (현재)
const vaspAdapter = new ExternalVASPAdapter({
  baseUrl: process.env.VASP_BASE_URL,
  apiKey: process.env.VASP_API_KEY,
});

// Phase 3 (교보 VASP 인가 후)
const vaspAdapter = new KyoboVASPAdapter({
  hsmEndpoint: process.env.HSM_ENDPOINT,
  mpcConfig: process.env.MPC_CONFIG,
});
```

변경 불필요

- IssuerService 비즈니스 로직
- TxStateMachineService 상태 머신
- VaspTxClientAdapter
- 모든 테스트 (인터페이스 Mock 사용)

KyoboVASPAdapter (2/2) — Phase 3 내재화 구현 항목

HSM · TRAVEL RULE · AML · APPROVAL WORKFLOW

① HSM/MPC 기반 지갑 생성·서명

- Private key가 서버 메모리에 존재하지 않는 구조 필수
- 금융보안원 전자금융기반시설 보안 요건 준수
- MPC: 키 분산 — 단일 서버 해킹 시 키 유출 없음

② Travel Rule 자체 처리

- TRISA 또는 VerifyVASP 프로토콜 직접 구현
- 상대방 VASP와 암호화 채널로 수신자 정보 교환
- Phase 1: VASP가 처리 → Phase 3: 직접 처리

③ AML 자체 스크리닝

- Chainalysis 또는 Elliptic API 직접 연동
- 블랙리스트 DB 자체 유지
- Phase 1: VASP가 스크리닝 → Phase 3: 직접 제어

④ 출금 한도·승인 워크플로우

- 대규모 출금 다중 서명 승인 (M-of-N)
- 이상 거래 탐지 → 자동 동결
- Phase 1: VASP가 한도 관리 → Phase 3: 자체 정책

인가 전제 조건 — 금융위원회 VASP 신고·인가 취득 선행. 일정 미확정. 기술 구현 준비는 지금부터.

IBlockchainAdapter vs IVASPSAdapter

TWO STRATEGY PATTERNS — SAME DESIGN PRINCIPLE

구분	IBlockchainAdapter	IVASPSAdapter
선언 위치	chain-adapters/interfaces/	vasp/interfaces/
Phase 1 구현체	EVMAAdapter read-only	ExternalVASPSAdapter write 위탁
Phase 3 구현체	EVMAAdapter write 활성화	KyoboVASPSAdapter 내재화
역할	블록체인 RPC 추상화	TX 위탁 · 지갑 관리 추상화
write 권한	Phase 3에서 활성화 (privateKey 주입)	Phase 1부터 외부 위탁으로 처리
교체 방법	DI 교체 — 상위 레이어 무변경	DI 교체 — 상위 레이어 무변경

공통점 — Strategy Pattern

- 둘 다 Strategy Pattern
- 상위 레이어(IssuerService, TxStateMachineService)는 구현체를 모름
- Phase 전환 = DI 바인딩 교체 1줄

차이점 — write 경로

- IBlockchainAdapter: Phase 1은 read-only, Phase 3에서 직접 서명·전송
- IVASPSAdapter: Phase 1도 write — 단 **외부 VASPS에 위탁**
→ 서명 키가 교보 서버에 없음 = 규제 리스크 ↓